How-To Library

Informatica

Integrating Data Engineering Integration on the AWS Platform with Databricks and Delta Lake

© Copyright Informatica LLC 2020, 2022. Informatica and the Informatica logo are trademarks or registered trademarks of Informatica LLC in the United States and many jurisdictions throughout the world. A current list of Informatica trademarks is available on the web at https://www.informatica.com/trademarks.html.

Abstract

Customers of Amazon Web Services and Informatica can integrate Data Engineering Integration 10.4 and 10.5 with a Databricks compute cluster and Delta Lake storage resources in the AWS cloud environment. The integration allows users to run mappings and workflows on Databricks to access data from and write data to Delta Lake tables.

This article instructs administrators how to integrate Data Engineering Integration with Databricks in the AWS environment. The article also instructs developers how to create mappings and workflows to access data in Delta Lake tables hosted in the same, or another, AWS environment.

Supported Versions

- Data Engineering Integration 10.4, 10.5.0, 10.5.1, 10.5.2

Table of Contents

Databricks Integration Overview. . . . 3
Run-time Process on the Databricks Spark Engine. . . . 3
Databricks Environment. . . . 4
Databricks Integration Task Flow. . . . 4
Before You Begin Databricks Integration. . . . 5
Read the Release Notes. . . . 5
Verify System Requirements 5
Configure Preemption for Concurrent Jobs. . . . 6
Configure Access to Delta Lake Tables 7
Create a Staging Directory for Binary Archive Files. . . . 9
Create a Staging Directory for Run-time Processing. . . . 10
Prepare a Databricks User for Token Authentication. . . . 11
Configure the Data Integration Service. . . . 13
Install Python Libraries 13
Databricks Integration Tasks. . . . 13
Create a Databricks Cluster Configuration. . . . 14
Importing a Databricks Cluster Configuration from the Cluster. . . . 14
Importing a Databricks Cluster Configuration from a File. . . . 16
Cloud Provisioning Configuration 17
Verify Prerequisites. . . . 18
Create the Cloud Provisioning Configuration 18
Complete Upgrade Tasks. . . . 19
Configure Databricks Clusters from Warm Pools. . . . 19
Mappings Overview 20
Mapping Execution Plans. . . . 21
Rules and Guidelines for Mappings on the Databricks Spark Engine. . . . 22
Sequence Generator Transformation Support for Databricks. . . . 22
Delta Lake Sources and Targets 22

Update Strategy Transformation Support for Databricks. 23
Databricks Schema Evolution. 23
Troubleshoooting. 27
Connections. 28
Manually Creating a JDBC Connection to Databricks Resources. 28
Databricks Connection Properties. 30

Databricks Integration Overview

Data Engineering Integration can connect to Databricks on Azure and on AWS. Databricks is an analytics cloud platform that you can use with Microsoft Azure cloud services or Amazon Web Services. Databricks incorporates the open-source Apache Spark cluster technologies and capabilities.

The Data Integration Service installs the binaries required to integrate the Informatica domain with the Databricks environment. The integration requires Informatica connection objects and cluster configurations. A cluster configuration is a domain object that contains configuration parameters that you import from the Databricks cluster. You then associate the cluster configuration with connections to access the Databricks environment.

Note: With the following exceptions, all of the functionality described in this article applies to Informatica 10.4 and later releases:

- Informatica 10.5 adds support for warm pool access and the Sequence Generator transformation.
- Informatica 10.5.2 adds support for Databricks schema evolution and custom parameters.

For more information about these features, see the *Data Engineering Integration User Guide*.

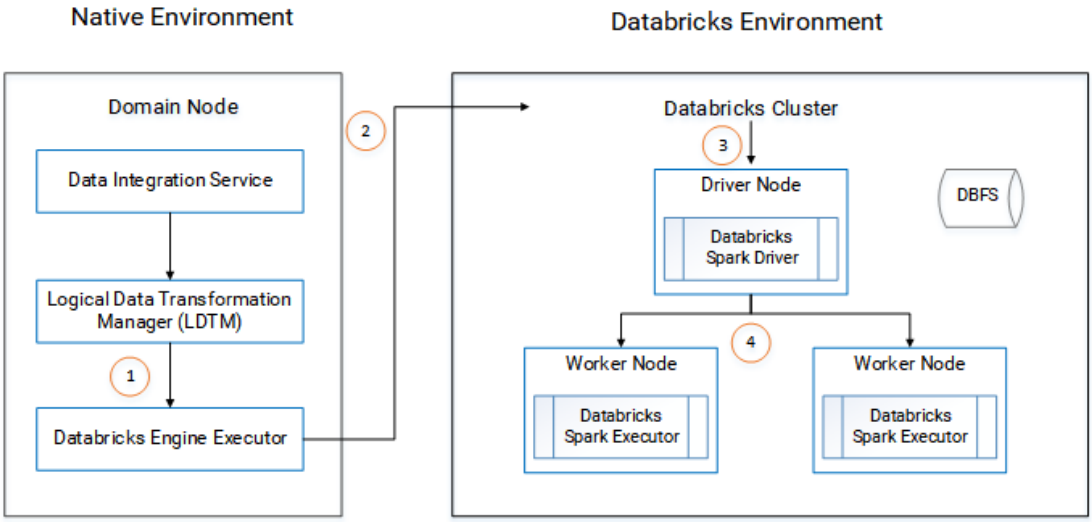
Perform the following tasks to integrate the Informatica domain with the Databricks environment:

1. Install or upgrade to the current Informatica version.
2. Perform pre-import tasks, such as verifying system requirements and permissions.
3. Import the cluster configuration into the domain.
4. Create a Databricks connection to run mappings within the Databricks environment.

Run-time Process on the Databricks Spark Engine

When you run a job on the Databricks Spark engine, the Data Integration Service pushes the processing to the Databricks cluster, and the Databricks Spark engine runs the job.

The following image shows the components of the Informatica and the Databricks environments:



1. The Logical Data Transformation Manager translates the mapping into a Scala program, packages it as an application, and sends it to the Databricks Engine Executor on the Data Integration Service machine.
2. The Databricks Engine Executor submits the application through REST API to the Databricks cluster, requests to run the application, and stages files for access during run time.
3. The Databricks cluster passes the request to the Databricks Spark driver on the driver node.
4. The Databricks Spark driver distributes the job to one or more Databricks Spark executors that reside on worker nodes.
5. The executors run the job and stage run-time data to the Databricks File System (DBFS) of the workspace.

Databricks Environment

Integration with the Databricks environment includes the following components:

Databricks Spark engine

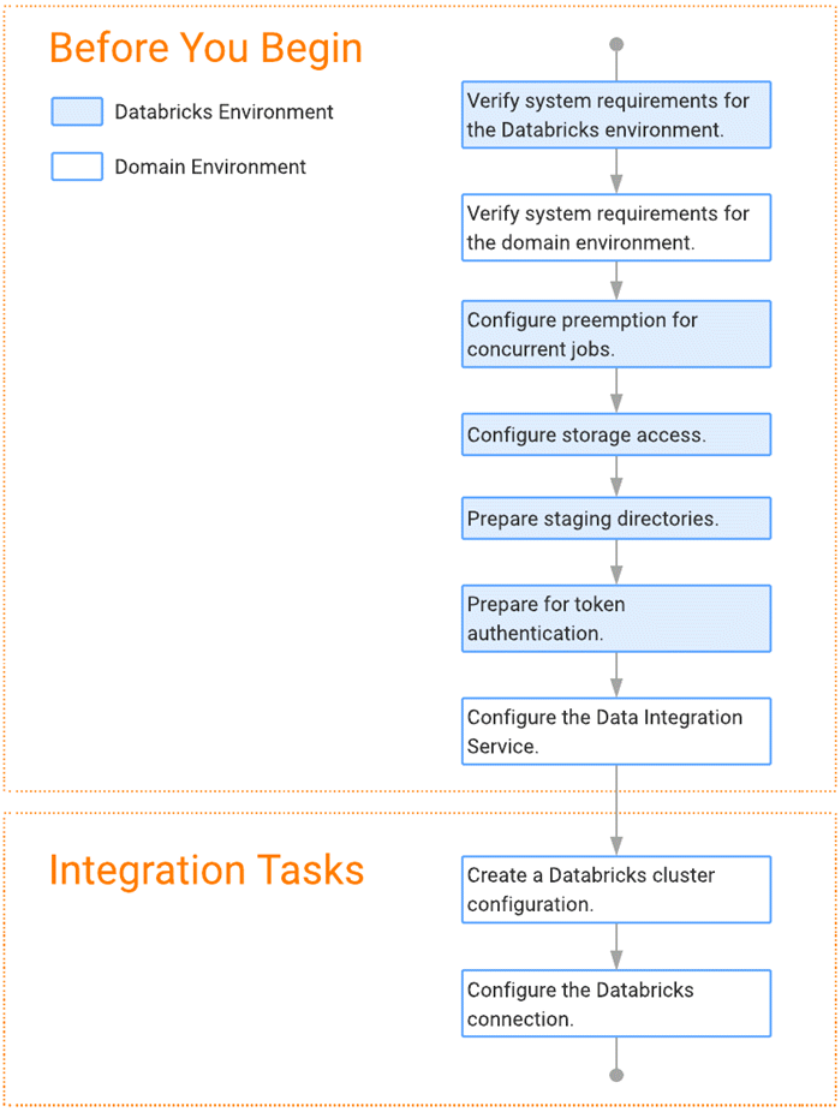
The Databricks run-time engine based on the open-source Apache Spark engine.

Databricks File System (DBFS)

A distributed file system installed on Databricks Runtime clusters. Run-time data is staged in the DBFS and is persisted to a mounted Blob storage container.

Databricks Integration Task Flow

The following diagram shows the task flow to integrate the Informatica domain with Databricks on AWS:



Before You Begin Databricks Integration

Perform the tasks in this section before you integrate Data Engineering Integration with the Databricks cluster.

Read the Release Notes

Read the Release Notes for updates to the installation and upgrade process. You can also find information about known and fixed issues for the release.

Find the Release Notes on the Informatica documentation portal.

Verify System Requirements

Verify that your environment meets the following minimum system requirements for the integration:

Informatica services

Install and configure the Informatica services and the Developer tool. Verify that the domain contains a Model Repository Service and a Data Integration Service.

Domain access to Databricks

Verify access domain access to Databricks through one of the following methods:

- The Informatica domain and Databricks are installed on the same AWS VPS.
- The Informatica domain and Databricks are installed on different AWS VPS's with peering enabled.

For more information about product requirements and supported platforms, see the Product Availability Matrix on Informatica Network:
https://network.informatica.com/community/informatica-network/product-availability-matrices

Configure Preemption for Concurrent Jobs

Configure the Databricks cluster to improve concurrency of jobs.

When you submit a job to Databricks, it allocates resources to run the job. If it does not have enough resources, it puts the job in a queue. Pending jobs fail if resources do not become available before the timeout of 30 minutes.

You can configure preemption on the cluster to control the amount of resources that Databricks allocates to each job, thereby allowing more jobs to run concurrently. You can also configure the timeout for the queue and the interval at which the Databricks Spark engine checks for available resources.

Configure the following Spark properties for the Databricks Spark engine:

spark.databricks.preemption.enabled

Enables the Spark scheduler for preemption. Default is false.

Set to: true

spark.databricks.preemption.threshold

A percentage of resources that are allocated to each submitted job. The job runs with the allocated resources until completion. Default is 0.5, or 50 percent.

Set to a value lower than default, such as 0.1.

spark.databricks.preemption.timeout

The number of seconds that a job remains in the queue before failing. Default is 30.

Set to: 1,800.

Note: If you set a value higher than 1,800, Databricks ignores the value and uses the maximum timeout of 1,800.

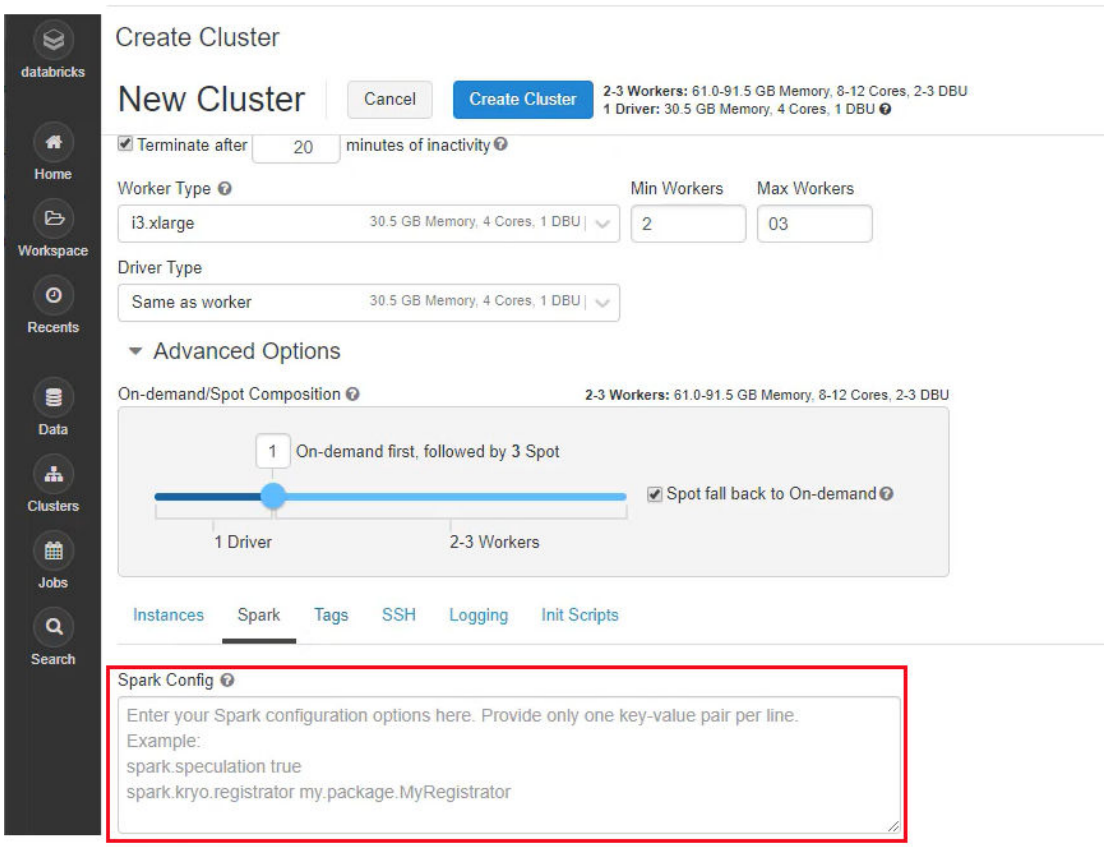
spark.databricks.preemption.interval

The number of seconds to check for available resources to assign to a job in the queue. Default is 5.

Set to a value lower than the timeout.

Changes take effect after you restart the cluster.

To configure Spark properties, from the Databricks console, select **Clusters** and select the Databricks cluster you want to integrate with Data Engineering. The following image shows the Advanced Options area where you configure Spark properties:



Important: Informatica integrates with Databricks, supporting standard concurrency clusters. Standard concurrency clusters have a maximum queue time of 30 minutes, and jobs fail when the timeout is reached. The maximum queue time cannot be extended. Setting the preemption threshold allows more jobs to run concurrently, but with a lower percentage of allocated resources, the jobs can take longer to run. Also, configuring the environment for preemption does not ensure that all jobs will run. In addition to configuring preemption, you might choose to run cluster workflows to create ephemeral clusters that create the cluster, run the job, and then delete the cluster.

Configure Access to Delta Lake Tables

To configure access for the domain to Delta Lake tables, download the Databricks JDBC driver to enable the connection between the domain and the Delta Lake resource, and configure Spark properties on the Databricks cluster to access Delta Lake tables on AWS resources

Download and Install the Databricks JDBC Driver

To use Delta Lake as a source or target, download the Databricks JDBC driver to enable the connection between the domain and the Delta Lake resource:

1. In a browser, go to the Databricks JDBC/ODBC driver page at online Databricks documentation. Fill out the form and submit it.
   After you submit the form, you receive an email message that includes multiple download options.
2. Save the JDBC driver.jar file to the following directory on each machine that hosts the Data Integration Service, and then restart the Informatica domain:
   `<Informatica home>/externaljdbcjars`

3. Save the JDBC driver .jar file to the following location on each machine that hosts the Developer tool client, and then restart the Developer tool:
   `<Developer tool home>/clients/externaljdbcjars`

Configure S3 and Redshift Storage Access

Storage access configuration depends on your choices for authentication and encryption.

Authentication Options

Configure the following authentication options:

Key Pair Authentication

If the data resources use AWS key pair authentication, add the access key and the secret key in the Spark tab of the Advanced Configuration section of the Databricks cluster configuration page. Separate each key value with a space.

The image below shows an example of the keys in the text entry pane of the Spark tab:



IAM Role Authentication

If the data resources use IAM role authentication, verify that the configuration meets one of the following requirements:

- The S3 bucket belongs to the AWS account in which the Databricks cluster resides.
- The S3 bucket belongs to a different AWS account than the one in which the Databricks cluster resides, and you enabled a cross-account policy to allow the cluster to access the bucket.

For more information about using IAM roles with a Databricks cluster, see the AWS documentation.

Encryption Options

Choose from the following types of encryption. Each can be combined with either of the two options for authentication.

Configure the following properties in the Spark configuration tab of the cluster:

Server-Side S3 Encryption (SSE-S3)

Configure the following property to enable SSE-S3 encryption:

```
spark.hadoop.fs.s3a.server-side-encryption-algorithm AES256
```

Note: If you use both key pair authentication and SSE-S3 encryption, then add this property in the Spark configuration tab after the first two lines for key pair authentication. For example:

```
spark.hadoop.fs.s3n.awsAccessKeyId YYYYXXXX
spark.hadoop.fs.s3n.awsSecretAccessKey XYYYYSSSSSSS
spark.hadoop.fs.s3a.server-side-encryption-algorithm AES256
```

Server-Side Encryption with KMS (SSE-KMS)

Configure the following properties to enable SSE-KMS encryption:

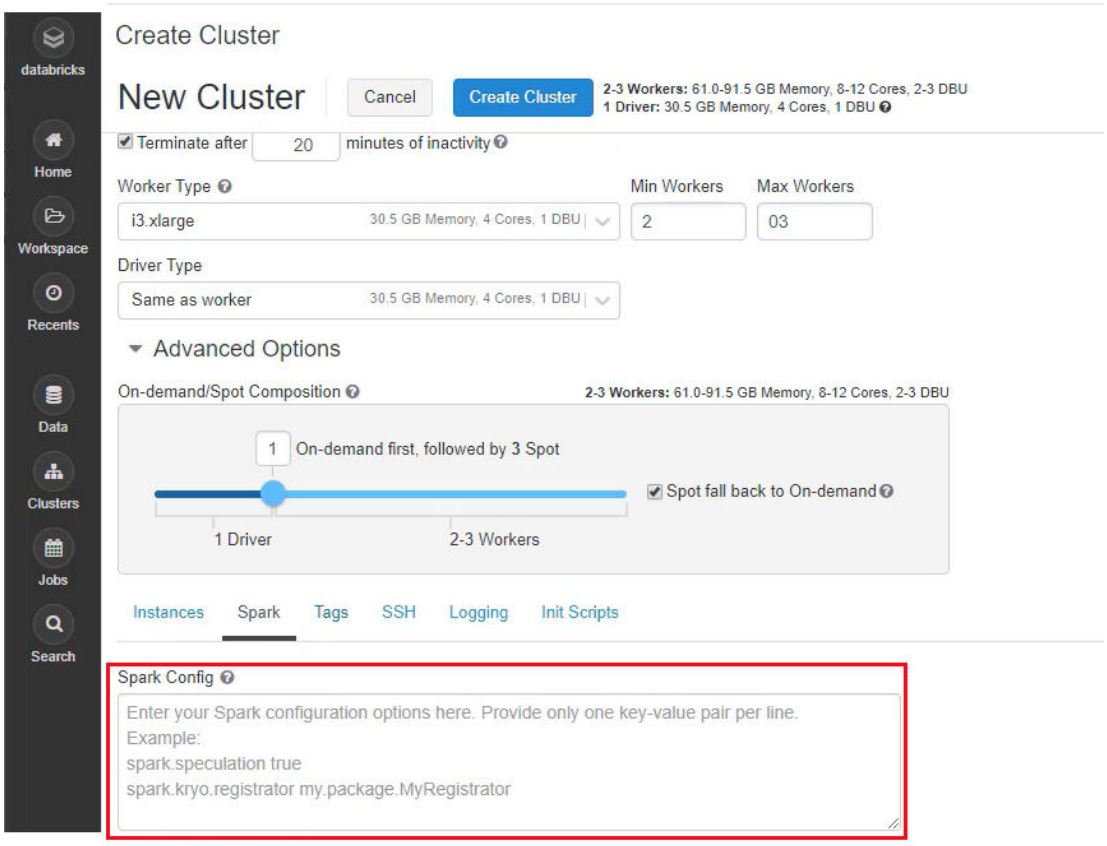
```
spark.hadoop.fs.s3a.server-side-encryption-kms-master-key-id arn:aws:kms:us-west-
XXXXX:key/XXXYYYYYYYYYYYYYYYYY
spark.hadoop.fs.s3a.server-side-encryption-algorithm aws:kms
spark.hadoop.fs.s3a.impl com.data bricks.s3a.S3AFileSystem
```

Note: If you use both key pair authentication and SSE-KMS encryption, then add these properties in the Spark configuration tab after the first two lines for key pair authentication. For example:

```
spark.hadoop.fs.s3n.awsAccessKeyld YYYYXXXX
spark.hadoop.fs.s3n.awsSecretAccessKey XYYYYSSSSSSS
spark.hadoop.fs.s3a.server-side-encryption-kms-master-key-id arn:aws:kms:us-west-
XXXXX:key/XXXYYYYYYYYYYYYYYYYY
spark.hadoop.fs.s3a.server-side-encryption-algorithm aws:kms
spark.hadoop.fs.s3a.impl com.data bricks.s3a.S3AFileSystem
```

See the Databricks documentation for examples on how to specify these settings in Scala or Python.

To configure access, from the Databricks Admin Console, select **Clusters** and select the Databricks cluster you want to integrate with Data Engineering. The following image shows the Advanced Options area where you configure the environment variables:



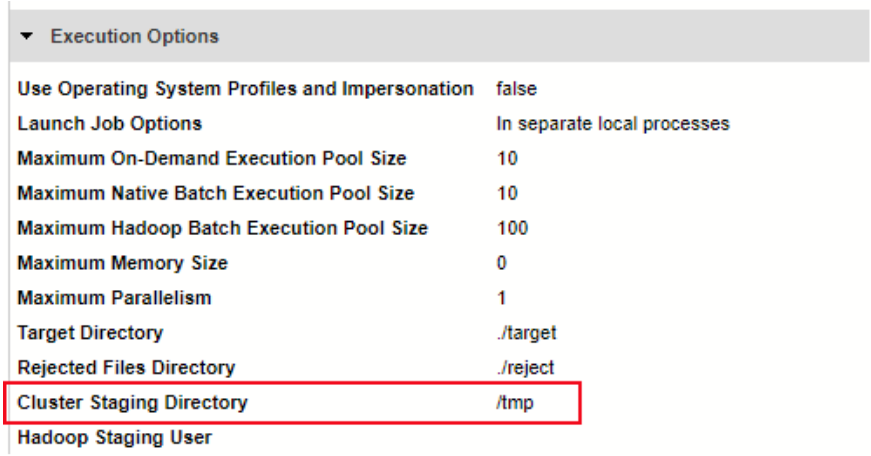
Create a Staging Directory for Binary Archive Files

Optionally, create a directory on DBFS that the Data Integration Service uses to stage the Informatica binary archive files.

By default, the Data Integration Service writes binary archive files to the DBFS directory `/tmp`.

Optionally, you can create a different staging directory for these files. If you create a staging directory, configure the path in the **Cluster Staging Directory** property of the Data Integration Service.

The following image shows where you configure the Cluster Staging Directory property in the Execution Options section of the Data Integration Service properties:



To create a directory on DBFS, see the Databricks documentation.

Create a Staging Directory for Run-time Processing

When the Databricks Spark engine runs a job, it stores temporary files in a staging directory.

Optionally, you can create a directory on DBFS to stage temporary files during run time. By default, the Data Integration Service uses the DBFS directory at the following path: `/<cluster staging directory>/DATABRICKS`

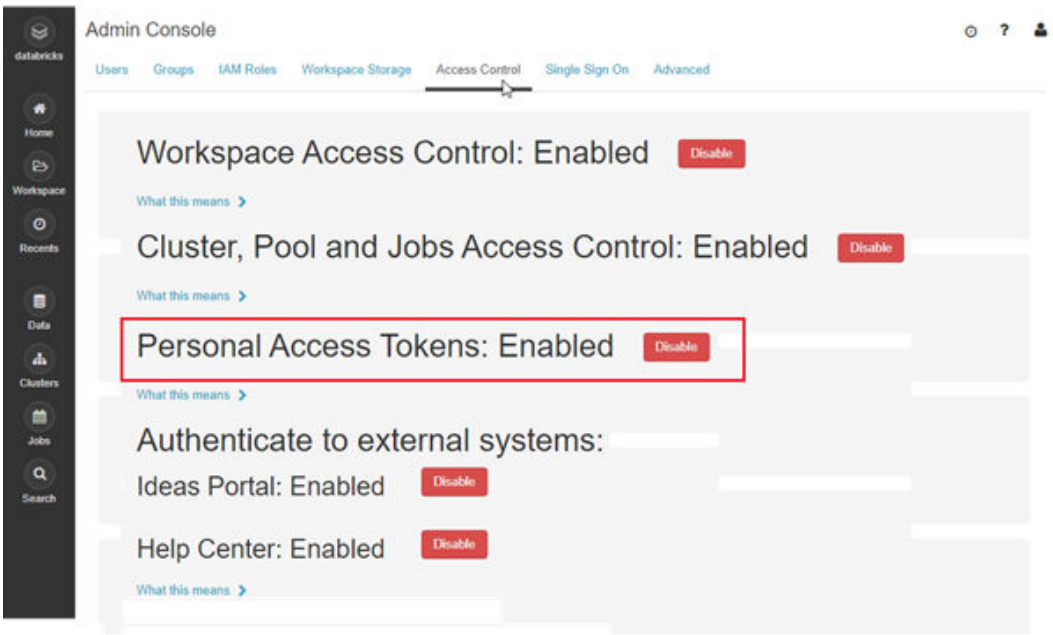
To create a directory on DBFS, see the Databricks documentation.

Prepare a Databricks User for Token Authentication

The Data Integration Service uses token-based authentication to provide access to the Databricks environment.

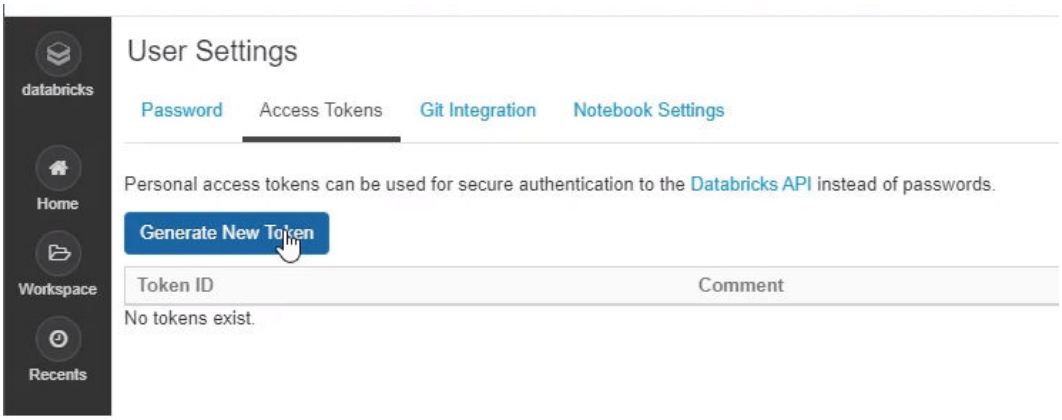
Create a Databricks user to generate the authentication token. Complete the following tasks to prepare for authentication.

1. In the Access Control tab of the Databricks Admin Console, verify that personal access tokens are enabled. The following image shows the access token setting:

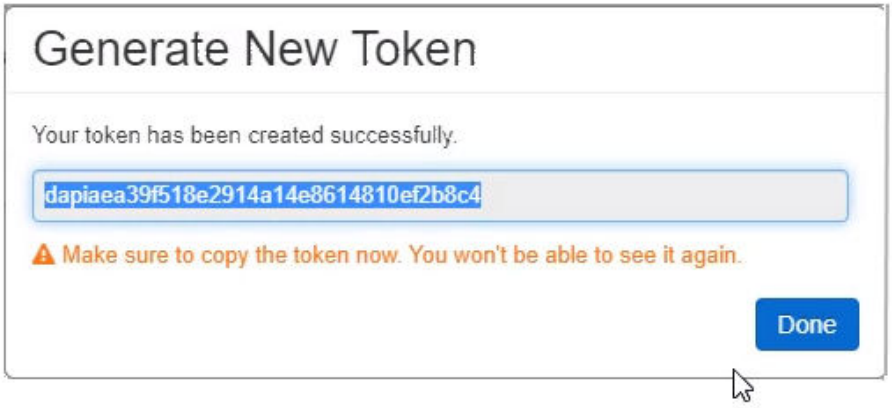


2. Verify that the Databricks environment contains a user to generate the token.
3. Grant permissions to the token user.
   - If you created staging directories, grant permission to access and write to the directories.
   - If you did not create staging directories, grant permission to create directories.
4. Log in to the Databricks Admin Console as the token user.

5. Go to **Access Control** > **User Settings** and click the **Generate New Token** button to generate a token for the user.
   The following image shows the **Generate New Token** button:



   The following image shows an example of the generated token:



   Important: The console displays the token only once, at generation. Copy and save the token value for later use.

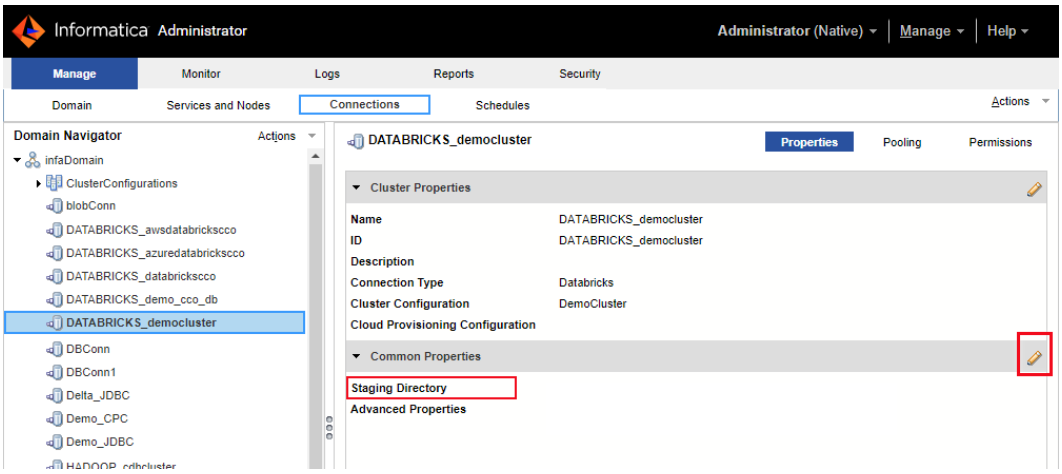
Configure the Data Integration Service

Configure the Data Integration Service to integrate with the Databricks environment.

Perform the following pre-integration tasks:

1. If you created a staging directory, configure the path in the Data Integration Service properties.
   The directory on the cluster where the Data Integration Service pushes the binaries to integrate the native and non-native environments and to store temporary files during processing. Default is `/tmp`.

   The following image shows the location in the Connections properties where you can configure the staging directory path:



2. Prepare an installation of Python on the Data Integration Service if you plan to run the Python transformation on a Hadoop cluster.

   Note: When you plan to run a mapping with a Python transformation on a Databricks cluster, it is not necessary to install Python on the Data Integration Service machine.

Install Python Libraries

Databricks comes installed with some Python libraries. If you need to install additional third-party Python libraries, use the pip installer for Databricks. The Databricks Spark engine supports Python version 3.

Perform the following tasks to install third-party Python libraries:

1. Write an init script that includes the Python libraries to install.
2. Upload the script to the DBFS directory.

Note: When you create an ephemeral workflow, include the init script file location in the advanced properties for the Create Cluster task.

For more information about the installed Python libraries that come with Databricks, refer to the Databricks documentation.

Databricks Integration Tasks

To integrate the Informatica domain with the Databricks cluster, you import cluster configuration settings to the domain.

You can choose to import the cluster configuration directly from the cluster. You can also save the configuration settings to a file, then import the file to the domain.

After the cluster configuration is imported to the domain, the Data Integration Service can push mapping logic to the Databricks cluster.

Create a Databricks Cluster Configuration

A Databricks cluster configuration is an object in the domain that contains configuration information about the Databricks cluster. The cluster configuration enables the Data Integration Service to push mapping logic to the Databricks environment.

Use the Administrator tool to import configuration properties from the Databricks cluster to create a cluster configuration. You can import configuration properties from the cluster or from a file that contains cluster properties. You can choose to create a Databricks connection when you perform the import.

Note:

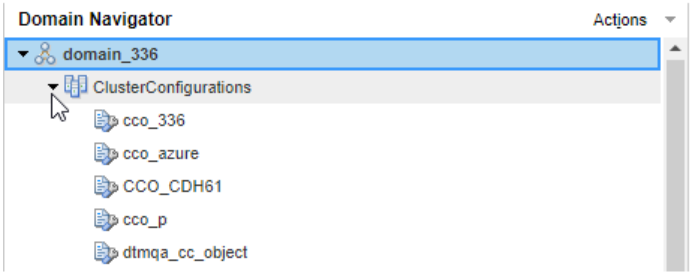
- Ensure that you integrate a Databricks cluster with only one Informatica domain.
- Ensure that a Databricks workspace has access to the database VM where you create the reference table.

Importing a Databricks Cluster Configuration from the Cluster

When you import the cluster configuration directly from the cluster, you provide information to connect to the cluster.

Before you import the cluster configuration, get cluster information from the Databricks administrator.

1. From the **Connections** tab, click the **ClusterConfigurations** node in the Domain Navigator.
   The following image shows the **ClusterConfigurations** node in the Domain Navigator on the **Connections** tab:

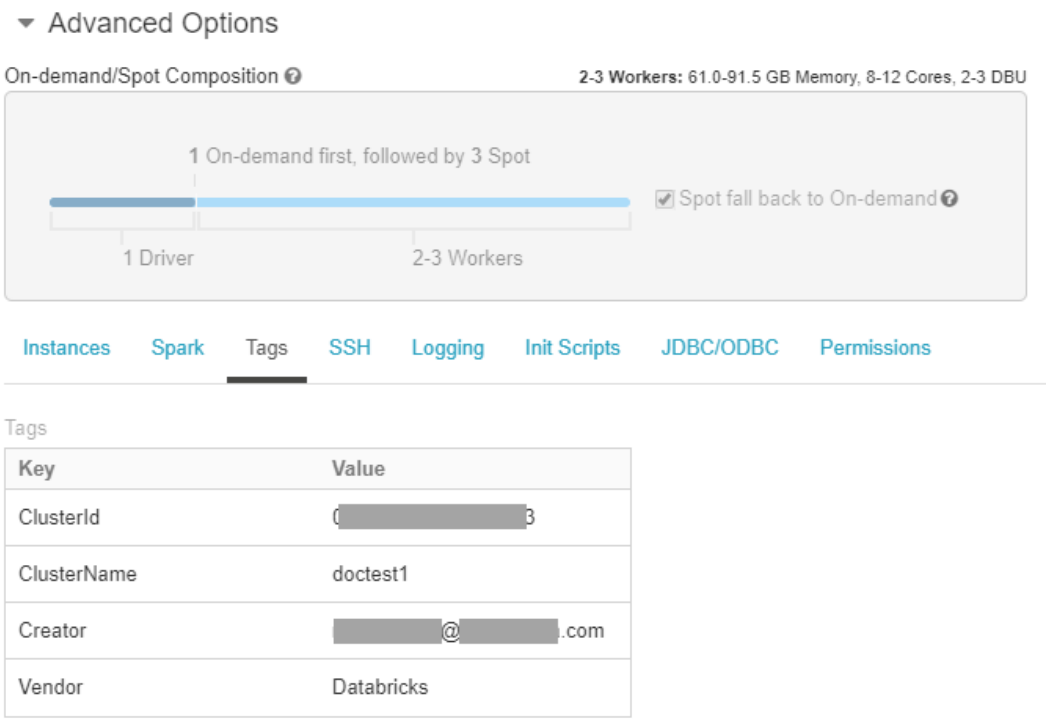


2. From the Actions menu, select **New** > **Cluster Configuration**.
   The **Cluster Configuration** wizard opens.
3. Configure the following properties:

Property	Description
Cluster configuration name	Name of the cluster configuration.
Description	Optional description of the cluster configuration.
Distribution type	The distribution type. Choose **Databricks**.
Method to import the cluster configuration	Choose **Import from cluster**.
Databricks domain	Domain name of the Databricks deployment.

Property	Description
Databricks access token	The token ID created within Databricks, required for authentication. . The token ID is visible only when it is created by the Databricks administrator. See "Prepare a Databricks User for Token Authentication" on page 11. **Note:** If the token has an expiration date, verify that you get a new token from the Databricks administrator before it expires.
Databricks cluster ID	The cluster ID of the Databricks cluster. To find the cluster ID on the Databricks portal, follow these steps: 1. Select **Clusters** from the object bar on the left side. 2. Select the cluster you want to integrate with the Informatica domain. 3. Click the **Spark ID** tab and expand the list of **Spark Properties**. 4. Select the **Tags** tab. See the image below.
Create connection	Choose to create a Databricks connection. If you choose to create a connection, the **Cluster Configuration** wizard associates the cluster configuration with the Databricks connection. If you do not choose to create a connection, you must manually create one and associate the cluster configuration with it. To manually create a connection, see the section about manually creating a connection in the Data Integration Engineering Administrator Guide.

To find the ClusterID, expand Spark **Advanced Properties** and then select the **Tags** tab. The following image shows the redacted ClusterID on the **Tags** tab:



4. Click **Next** to verify the information on the summary page.

Importing a Databricks Cluster Configuration from a File

You can import properties from an archive file to create a cluster configuration.

Complete the following tasks to import a Databricks cluster from a file:

1. Get required cluster properties from the Databricks administrator.
2. Create an .xml file with the cluster properties, and compress it into a .zip or .tar file.
3. Log in to the Administrator tool and import the file.

Create the Import File

To import the cluster configuration from a file, you must create an archive file.

To create the .xml file for import, you must get required information from the Databricks administrator. You can provide any name for the file and store it locally.

The following table describes the properties required to import the cluster information:

Property Name	Description
cluster_name	Name of the Databricks cluster.
cluster_ID	The cluster ID of the Databricks cluster.
baseURL	URL to access the Databricks cluster. This is the URL that appears in your browser menu bar when you access the Databricks cluster.
accesstoken	The token ID created within Databricks required for authentication.

Optionally, you can include other properties specific to the Databricks environment.

When you complete the .xml file, compress it into a .zip or .tar file for import.

Sample Import File

The following text shows a sample import file with the required properties:

```
<?xml version="1.0" encoding="UTF-8"?><configuration>
  <property>
    <name>cluster_name</name>
    <value>my_cluster</value>
  </property>
  <property>
    <name>cluster_id</name>
    <value>0926-294544-bckt123</value>
  </property>
  <property>
    <name>baseURL</name>
    <value>https://<AWS Databricks domain></value>
  </property>
  <property>
    <name>accesstoken</name>
    <value>dapicf76c2d4567c6sldn654fe875936e778</value>
  </property>
</configuration>
```

Import the Cluster Configuration

After you create the .xml file with the cluster properties, use the Administrator tool to import into the domain and create the cluster configuration.

1. From the **Connections** tab, click the **ClusterConfigurations** node in the Domain Navigator.
2. From the Actions menu, select **New** > **Cluster Configuration**.
   The **Cluster Configuration** wizard opens.
3. Configure the following properties:

Property	Description
Cluster configuration name	Name of the cluster configuration.
Description	Optional description of the cluster configuration.
Distribution type	The distribution type. Choose **Databricks**.
Method to import the cluster configuration	Choose **Import from file**.
Upload configuration archive file	The full path and file name of the file. Click the Browse button to navigate to the file.
Create connection	Choose to create a Databricks connection. If you choose to create a connection, the **Cluster Configuration** wizard associates the cluster configuration with the Databricks connection. If you do not choose to create a connection, you must manually create one and associate the cluster configuration with it.

4. Click **Next** to verify the information on the summary page.

Cloud Provisioning Configuration

A cloud provisioning configuration is an object in the domain that contains information about a cloud platform like AWS or Microsoft Azure. The cloud provisioning configuration gives the Data Integration Service the information it needs to create a cluster on the cloud platform.

Create a cloud provisioning configuration when you configure a cluster workflow for ephemeral clusters. You create a cluster workflow to automate the creation of clusters and workflow tasks on an Amazon Web Services, Microsoft Azure, or the Databricks compute cluster. For more information about cluster workflows, see the *Data Engineering Integration User Guide* .

The Data Integration Service uses the information in the cloud provisioning configuration to establish a relationship between the workflow Create Cluster task and the cloud platform, and to run tasks on the cluster that the workflow creates. Using authentication credentials from the cloud provisioning configuration, the Data Integration Service submits jobs to the compute cluster using the REST API.

For more information about the Databricks cluster Jobs API, see the Databricks documentation.

The cluster connection that the cluster workflow uses contains a reference to the cloud provisioning configuration.

Consider the following high-level process for using the cloud provisioning connection:

1. Verify prerequisites.

2. Create the cloud provisioning configuration.
3. Create a cluster connection for the workflow.

After you create the cloud provisioning configuration and the cluster connection, a developer uses the Developer tool to create, deploy, and run a cluster workflow. For information about creating and using cluster workflows to create ephemeral clusters, see the Data Engineering Integration User Guide.

Verify Prerequisites

Verify the following cloud platform and domain prerequisites.

Cloud Platform Prerequisites

Verify the following prerequisites for the AWS platform:

- Create a user account with administrator permissions on AWS.
- Create a virtual private cloud (VPC) on AWS, or identify an existing one. The Informatica domain, if it is deployed on AWS, and the cluster that the workflow creates must be located on this VPC.
- If the Informatica domain is installed on-premises, enable DNS resolution.

Domain Prerequisites

Verify the following prerequisites for the Informatica domain:

- An Informatica domain must be running on an EC2 instance on the AWS cloud platform, or on an on-premises machine.
- Configure the VPN to connect to the AWS VPC where the Databricks cluster is running.
- You must have permissions to create connections on the domain.

Enable DNS Resolution

If the Informatica domain is installed on-premises, you must enable DNS resolution over the VPN that connects the domain and the cloud platform.

To run mappings from an on-premises deployment of Data Engineering Integration to a cluster on AWS, you must install and configure Unbound on an EC2 instance. Unbound enables DNS resolution from an on-premises network. To read how to install and configure Unbound in the AWS VPC, see the AWS documentation.

Create the Cloud Provisioning Configuration

Create the cloud provisioning configuration and configure it with information that the domain needs to access and create resources on the cloud platform.

The properties to configure in the cloud provisioning configuration depend on the cloud platform.

1. From the **Connections** tab, right-click the Domain node and select **New** > **Connection**.

   The **New Connection** dialog box opens.
2. Enter property values in the configuration wizard, then click **Finish** to create the cloud provisioning configuration.

The following table describes the Databricks cloud provisioning configuration properties:

Property	Description
Name	Name of the cloud provisioning configuration. **Tip:** Because the Administrator tool lists cloud provisioning configuration objects with other connections, use a naming convention such as "CPC" as part of the name of the object to help identify it.
ID	The cluster ID of the Databricks cluster.
Description	Optional description of the cloud provisioning configuration.
Databricks domain	Domain name of the Databricks deployment.
Databricks token ID	The token ID created within Databricks required for authentication. **Note:** If the token has an expiration date, verify that you get a new token from the Databricks administrator before it expires.
Advanced Properties	Advanced properties that are unique to the Databricks cloud provisioning configuration.

The cloud provisioning configuration appears in the list of connections in the **Domain Navigator**.

Complete Upgrade Tasks

If you upgraded the Informatica platform, you need to perform some additional tasks within the Informatica domain.

Based on the version that you upgraded from, you might need to perform the following updates:
Update date/time data format for sources and targets

Perform this task in the following situation:
- You upgraded from version 10.2.2 or earlier.

Effective in version 10.4.0, the Databricks Spark engine reads and writes date/time values in the format YYYY-MM-DD HH24:MM:SS.US.

To continue to use the Databricks Spark engine to read date/time data in other formats, edit mappings to include an Expression transformation. Read the date/time field from the source as a string. In the expression, convert the string to a date/time format.

To write data to a different format, use an Expression transformation to convert the date/time data to a different format. Then write the data to the target as a string.

Configure Databricks Clusters from Warm Pools

Effective in Informatica 10.5, on the Azure and AWS Databricks platforms, you can configure Databricks ephemeral clusters to start on sets of cached VM instances, called warm pools. The VM instances wait on standby in a running state for ephemeral cluster creation. When you enable warm pools, you reduce the time it takes to create clusters if the pool contains enough idle instances.

For more information about using cluster workflows to create ephemeral clusters, see the *Data Engineering Integration User Guide* .

Prerequisite

Ephemeral clusters from warm pools require the Databricks administrator to create a cluster pool in the Databricks workspace. Choose from any available instance types for the VMs in the pool.

Configure the following properties when you create pools:

Property	Description
Min Idle	Minimum number of instances that will remain running in the pool in an idle state. These instances do not terminate, regardless of the setting specified in the Idle Instance Auto Termination property.
Idle Instance Auto Termination	The number of minutes that the pool will maintain an instance in an idle state after jobs are complete.

Note: The idle instances in the pool do not terminate regardless of the value in the Idle Instance Auto Termination property. Amazon charges for idle instances in a pool.

Configure the Cluster Workflow to Use Warm Pool Instances

Enable warm pools for Databricks ephemeral clusters in the Create Cluster task Advanced Properties.

The following table describes the properties to enable warm pools:

Property	Description
Warm Pools	Check this option to enable creation of the Databricks cluster using warm pools. Default is unchecked (false).
Pool ID	Required when you enable warm pools. Value of the DatabricksInstancePoolId property that identifies the warm pool resource.

Note: When you enable warm pools, some of the advanced properties that allow you to specify cluster instance details are disabled. Set cluster instance details when you configure pools in the Databricks console.

Mappings Overview

Configure the run-time environment in the Developer tool to optimize mapping performance and process data that is greater than 10 terabytes. The Data Integration Service pushes the processing to the Databricks cluster.

You can run standalone mappings and mappings that are a part of a workflow.

To run mappings on Databricks, you must configure a connection to access the environment. You can set the run-time properties for the environment and for the Databricks engine that runs the mapping. You can view the execution plan for a mapping.

The Data Integration Service logs messages from the Databricks Spark engines and the DTM.

Mapping Execution Plans

The Data Integration Service generates an execution plan to run mappings on the Databricks Spark engine. The Data Integration Service translates the mapping logic into code that the run-time engine can execute. You can view the plan in the Developer tool before you run the mapping and in the Administrator tool after you run the mapping.

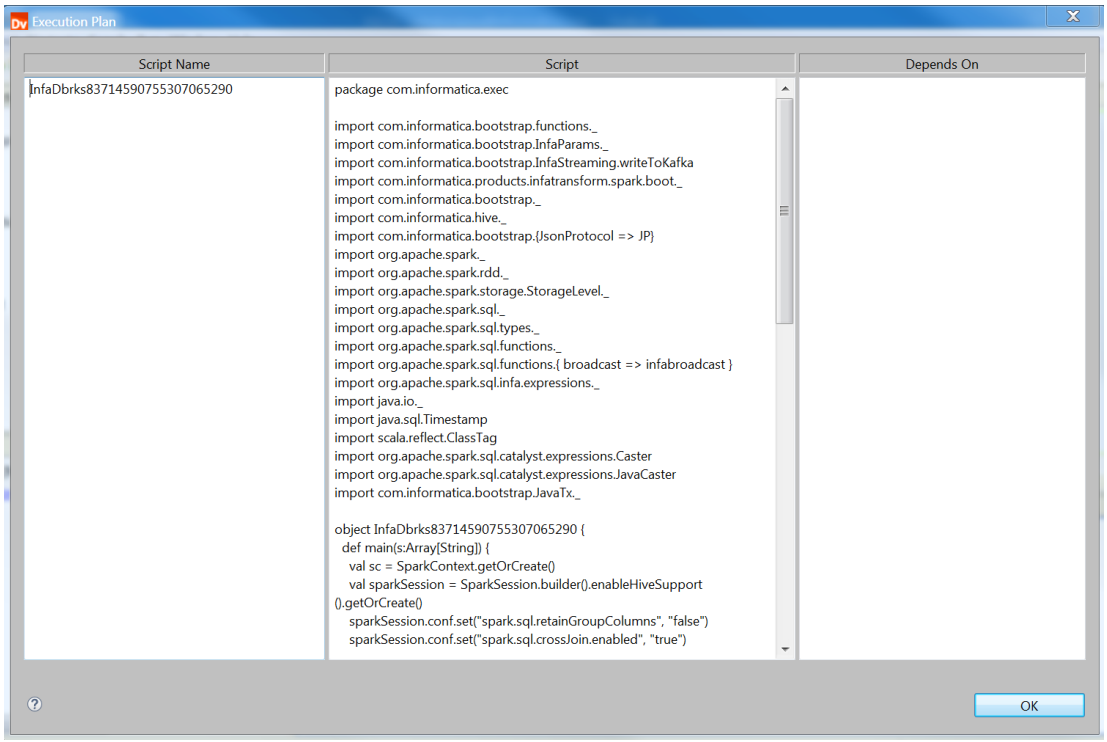
The Databricks Spark execution plan shows the run-time Scala code that runs the mapping logic. A translation engine translates the mapping into an internal representation of the logic. The internal representation is rendered into Scala code that accesses the Spark API. You can view the Scala code in the execution plan to debug the logic.

Databricks Spark Engine Execution Details

You can view the details of a Databricks Spark engine execution plan from the Administrator tool or Developer tool.

The Databricks Spark engine execution plan shows the Scala code to run on the Databricks Spark engine.

The following image shows the execution plan for a mapping to run on the Databricks Spark engine:



The Databricks Spark engine execution plan has the following details:

- Script ID. Unique identifier for the Databricks Spark engine script.
- Script. Scala code that the Data Integration Service generates based on the mapping logic.
- Depends on. Tasks that the script depends on. Tasks include other scripts and Data Integration Service tasks.

Viewing the Execution Plan

You can view the engine execution plan for a mapping. You do not have to run the mapping to view the execution plan in the Developer tool.

Note: You can also view the execution plan in the Administrator tool.

1. In the Developer tool, open the mapping.

2. Right-click the editor and select **Show Execution Plan**.

   The **Show Execution Plan** dialog box opens.

3. Select the mapping configuration and select **OK**.

   The Developer tool shows the details for the execution plan.

Rules and Guidelines for Mappings on the Databricks Spark Engine

A non-native environment like Databricks Spark uses distributed processing and processes data on different nodes. Each node does not have access to the data that is being processed on other nodes. As a result, the runtime engine might not be able to determine the order in which the data originated. So, when you run a mapping in a non-native environment and you run the same mapping in the native environment, both mappings return correct results, but the results might not be identical. For more information, see the Developer Transformation Guide.

Consider the following run-time differences on the Databricks Spark engine:

- When you use the auto optimizer level, the early selection optimization method is enabled if the mapping contains any data source that supports pushing filters to the source on Spark or Databricks Spark engines. For more information about optimizer levels, see the *Informatica Developer Mapping Guide*.
- Databricks Spark performs auto-optimization on jobs based on configuration settings for the cluster. If you set optimization through the Spark.default.parallelism property, the Databricks Spark engine ignores this setting. As a consequence, it is not possible to configure optimization on the job level.

Sequence Generator Transformation Support for Databricks

Effective in Informatica 10.5, you can run Sequence Generator transformations on the Databricks Spark engine.

The Sequence Generator transformation is supported when the Databricks cluster and the Informatica domain reside in the same virtual network. For more information about using a Sequence Generator transformation in mappings, see the *Developer Transformation Guide*.

Delta Lake Sources and Targets

Delta Lake is a data lake resource that stores data in large tables. Databricks uses proprietary Delta software to manage stored data and allow fast access to the data. Delta Lake supports ACID transactions.

Mappings can access managed Databricks tables, including Delta Lake tables. Delta Lake is a data lake resource that stores data in large tables. Databricks uses proprietary Delta software to manage stored data and allow fast access to the data. Delta Lake supports ACID transactions.

You can use the schema evolution feature of Databricks in dynamic mappings. Mappings succeed with merge enabled in the mapping and schema enforcement disabled in the target. See "Databricks Schema Evolution" on page 23.

To use Delta Lake resources in a mapping, create a relational data object and use a JDBC connection to connect to the resource. To perform data ingestion to a Delta Lake target, use a pass-through mapping with a flat file source.

Note: Mappings that access Delta Lake tables must use the Databricks Spark engine. If you run a Delta Lake mapping in the native environment with the JDBC connection, the mapping succeeds, but no data is written to the target.

Update Strategy Transformation Support for Databricks

You can use the Update Strategy transformation to perform INSERT, UPDATE, and other operations. You can also perform INSERT and UPDATE operations in select Delta Lake columns.

General Restrictions

You can only use the Update Strategy transformation on the Databricks Spark Engine when you use a Delta Lake table target.

The Update Strategy transformation is supported only on Databricks distributions that support Delta ACID.

Mapping Validation

Mapping validation fails in the following situations.

- The Update Strategy transformation is connected to more than one target.
- The Update Strategy transformation is not located immediately before the target.
- The Update Strategy transformation target is an external ACID table.
- The target does not contain a connected primary key.
- The property to enable truncation of the target table at run time is selected.
- One of the following target strategies for the target table at run time is selected:
  - Create or replace the target table
  - ApplyNewColumns
  - ApplyNewSchema
  - Fail

Databricks Schema Evolution

Schema enforcement and evolution enable you to manage changes in the Databricks table schema. You can choose different strategies for managing schema changes.

Schema enforcement monitors changes to Databricks table schemas and rejects changes that do not match the target table schema. When Databricks rejects changes, it cancels the write transaction and logs an exception. If you determine that you want to incorporate new columns in the target, schema evolution enables you to add them to the target in a controlled fashion. Schema evolution is also known as schema validation.

To use schema evolution, you need to disable schema enforcement in the target Databricks workspace.

For more information about Databricks schema enforcement and evolution, see the Databricks documentation.

Use the Developer tool to apply schema evolution to a mapping and update Databricks target tables.

You can use the following strategies to enable schema evolution:

- CREATE. When you configure mappings to use the CREATE strategy, you drop the existing target table and replace it with a target table that uses the new schema.
  The mapping drops the existing target table and all its existing data and recreates the target table with all the columns from the source and their data. Thereafter, every time the mapping runs, the mapping drops the target table with its data and recreates the target table from the source schema.
- RETAIN using pre-SQL. When you configure mappings to use the RETAIN strategy, you use a pre-SQL statement to add a subset of the schema's changed columns to the existing target table. Any existing target data is also retained. You can parameterize the pre-SQL query, or substitute a parameter set.

- CREATE using post-SQL. When you configure mappings to sue the CREATE strategy with a post-SQL statement, you add a subset of the schema's changed columns to the existing target table. Any existing target data is also retained. The mapping loads source data from selected new columns into a temp table, and then the Post-SQL statement merges it into the existing target table schema. You can parameterize the post-SQL query, or substitute a parameter set.

Note: You can't use schema evolution in the following conditions:

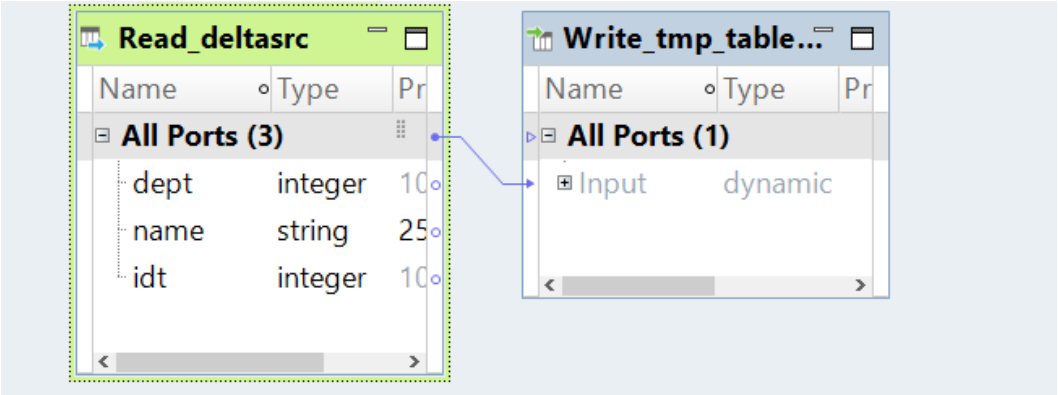
- When table access control is enabled.
- With `INSERT INTO` or `.write. insertInto()` clauses in Pre-SQL or Post-SQL.

Using the CREATE strategy

Use the CREATE strategy in a mapping to drop the existing Delta Lake target table, including its data, and create a new target table with all source schema columns and their data.

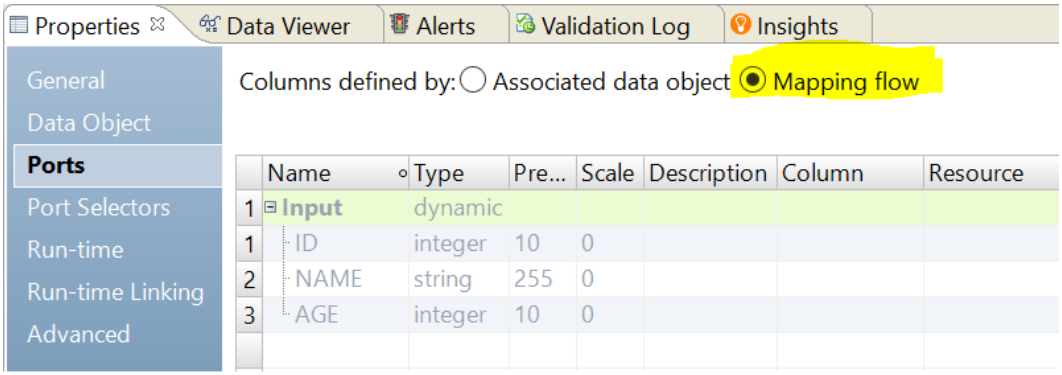
1. Create a passthrough mapping with ports dynamically linked from the Read transformation to the Write transformation.

   The following image shows the linked transformations:



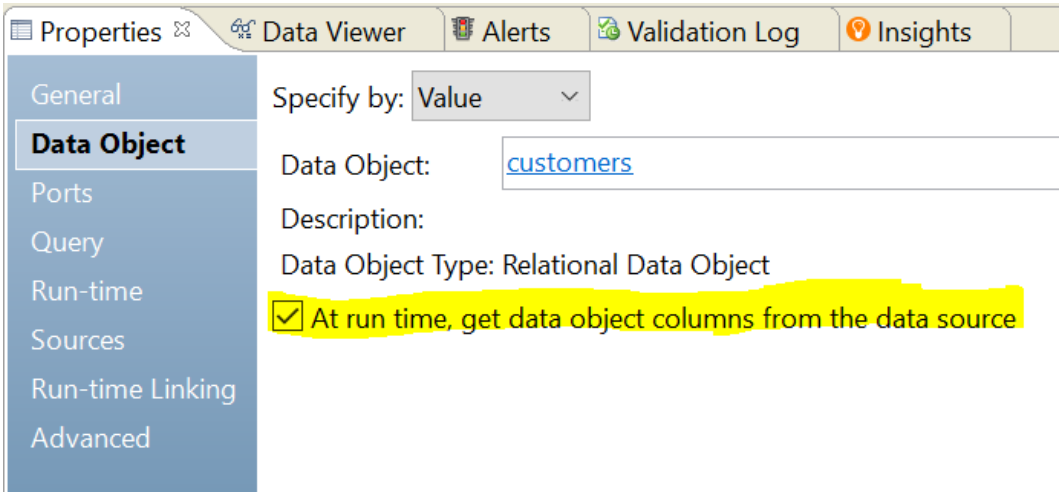
2. Select the Write transformation and the Ports property sheet. Select the **Columns defined by mapping flow** option.

   The following image shows the location of the **Mapping flow** option in the Ports properties:



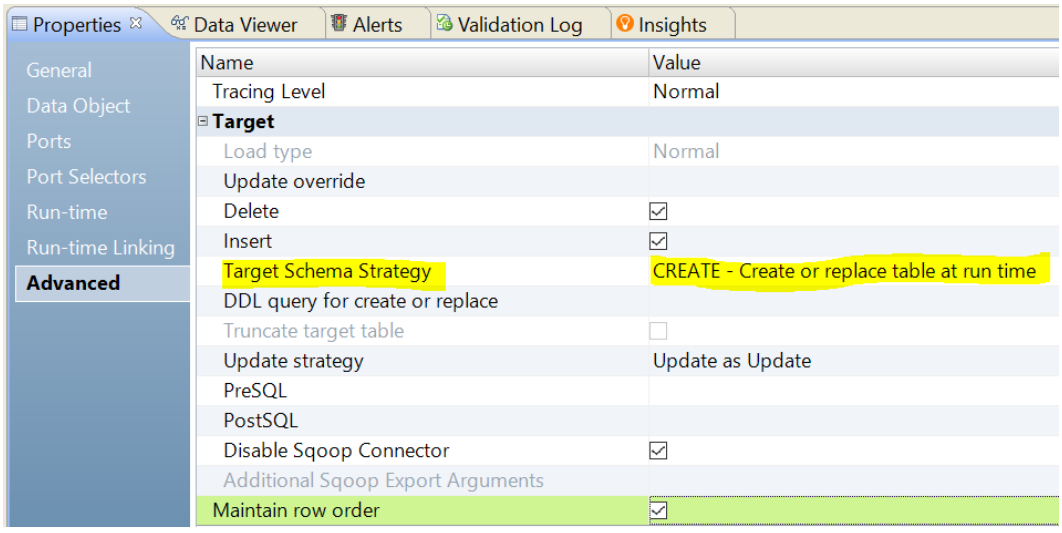
3. Select the Read transformation and the Data Object property sheet. Select the **At run time, get data object columns from the data source** option.

   The following image shows the option location in the Data Object properties:



4. Select the Write transformation and the Advanced property sheet. For the Target Schema Strategy property, select the **CREATE** option.

   The following image shows the **Target Schema Strategy** property in the Advanced properties:



Using the RETAIN Strategy with Pre-SQL

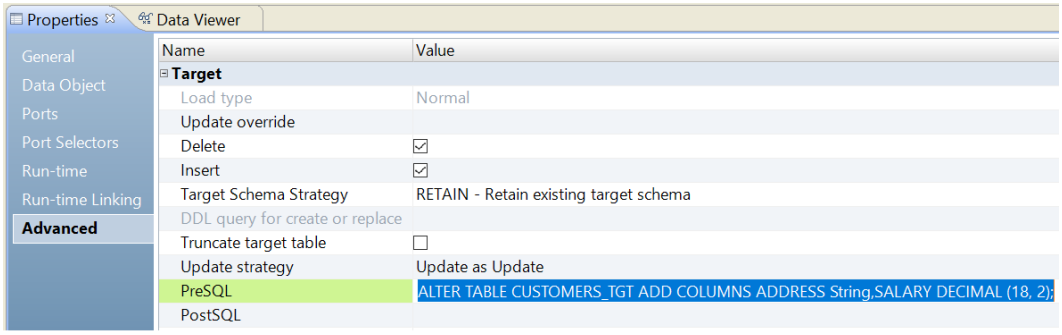
Use pre-SQL with the RETAIN strategy in a mapping to retain the existing Delta Lake target table, including its data, and add columns to it.

1. Create a passthrough mapping with ports dynamically linked from the Read transformation to the Write transformation.
2. Select the Write transformation and the Advanced property sheet. For Target Schema Strategy, select **Retain.**

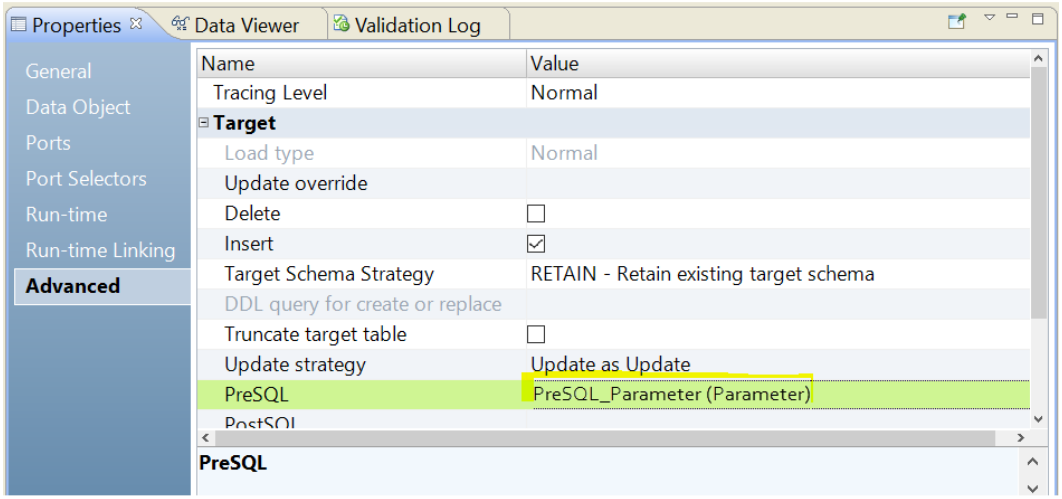
3. Open the Advanced property sheet. Use an ALTER TABLE Pre-SQL statement to specify the columns from the source schema to add to the target table schema.

   You can choose to put the Pre-SQL query itself into the PreSQL property, or use a parameter or parameter set that contains the query. For more information about using parameterization in mappings, see the *Informatica Developer Tool Guide* and the *Informatica Developer Mapping Guide*.

   The following image shows the PreSQL property location in the Advanced properties:



   The following image shows the PreSQL property with a parameter as a value:



Using the CREATE Strategy with Post-SQL

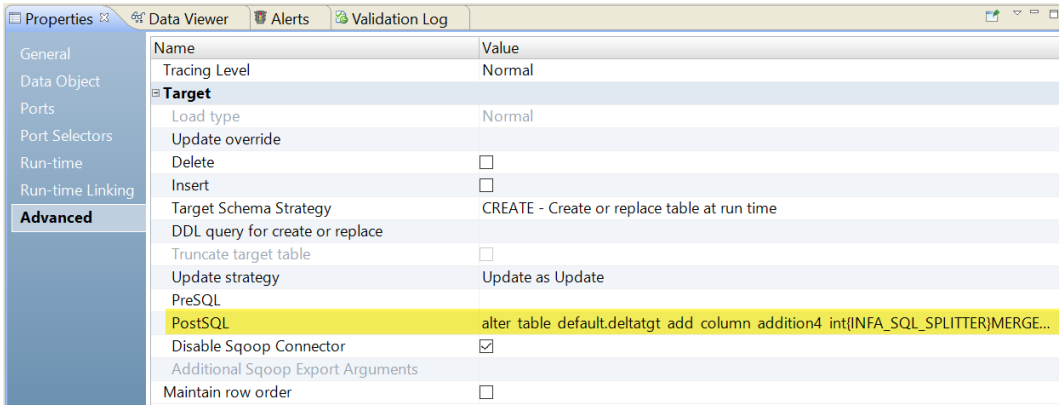
Use the CREATE strategy and a post-SQL statement to add a subset of the source schema's new columns to the existing target table.

1. Create a passthrough mapping with ports dynamically linked from the Read transformation to the Write transformation.
2. Select the Write transformation and the Advanced property sheet. For Target Schema Strategy, select **Create.**
3. In the Update Strategy property, select **Update as Update.**

4. In the PostSQL property, use an ALTER TABLE command to specify the columns to add. See the example below.

   You can choose to put the Post-SQL query itself into the PostSQL property, or use a parameter or parameter set that contains the query. For more information about using parameterization in mappings, see the *Informatica Developer Tool Guide* and the *Informatica Developer Mapping Guide*.

   The following image shows the PostSQL statement in the Advanced properties:



Example

Use the SQL statement below as a template for the post-SQL statement.

```
ALTER TABLE default.deltatgt add column addition4 int{INFA_SQL_SPLITTER}MERGE INTO
default.deltatgt USING default.tmp_table_auto3 ON default.deltatgt.idt =
default.tmp_table_auto3.idt WHEN MATCHED THEN UPDATE SET default.deltatgt.addition4 =
default.tmp_table_auto3.addition4 WHEN NOT MATCHED THEN INSERT (dept, name, idt, addition,
addition2, addition3, addition4) VALUES (dept, name, idt, addition, addition2, addition3,
addition4)
```

Troubleshoooting

Consider the following troubleshooting tips when you run mappings on the Azure cluster:

The VM fails to find a script extension file

When the mapping uses a script extension, the mappings fails with an error like:

```
Enable failed: processing file downloads failed: failed to download file[0]: failed to
download the file: http request failed: Get [REDACTED] dial tcp 13.107.246.10:443: i/o
timeout
```

The mapping fails because access to the script is blocked by the outbound firewall.

Workaround: Add the following ports and IP addresses to the outbound firewall rule:

Ports	IP Addresses/Domains
80 and 443	`catalogartifact.azureedge.net`
All	`VirtualNetwork`
443	`Storage`
123	`0.rhel.pool.ntp.org`

Ports	IP Addresses/Domains
123	1.rhel.pool.ntp.org
123	2.rhel.pool.ntp.org
123	3.rhel.pool.ntp.org
443	13.91.47.76/32
443	40.85.190.91/32
443	52.187.75.218/32
443	52.174.163.213/32
443	52.237.203.198/32
443	download.postgresql.org
443	download.postgresql.org
443	download.postgresql.org
443	download.postgresql.org

Connections

Databricks connections contain information to connect to the Databricks cluster. When you imported the cluster configuration, you made a choice whether to automatically create a Databricks connection at the same time. If you did not create the connection at that time, you must create it manually to integrate the domain and the Databricks cluster.

To connect to a Delta Lake resource, you must also create a JDBC connection.

Note: You must configure both the Databricks connection and the JDBC connection to access Delta Lake resources and run mappings on a Databricks cluster.

This section tells how to manually create a JDBC connection. It also contains descriptions of the Databricks connection properties to use in creating a Databricks connection if you did not previously create one.

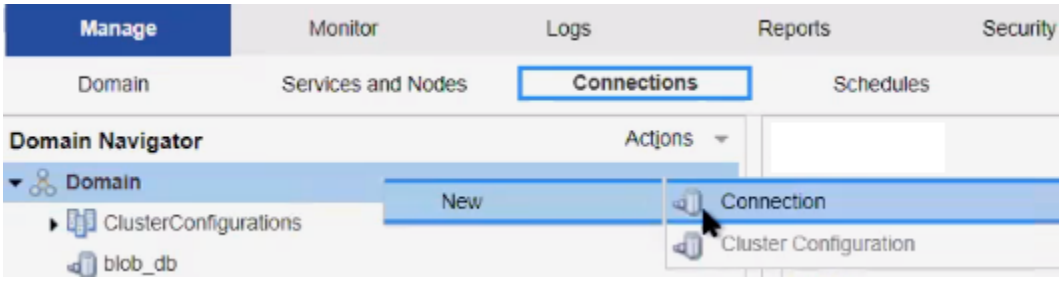
Manually Creating a JDBC Connection to Databricks Resources

To enable the domain to access Databricks resources in the cloud platform environment, you must manually create and configure a JDBC connection.

You can use the Informatica Administrator or Developer tools to create a connection. The following steps tell how to create a JDBC connection using the Administrator tool.

1. In the Administrator tool, click the **Connections** tab.
2. Select the domain, click the Actions menu, and select **New** > **Connection**.

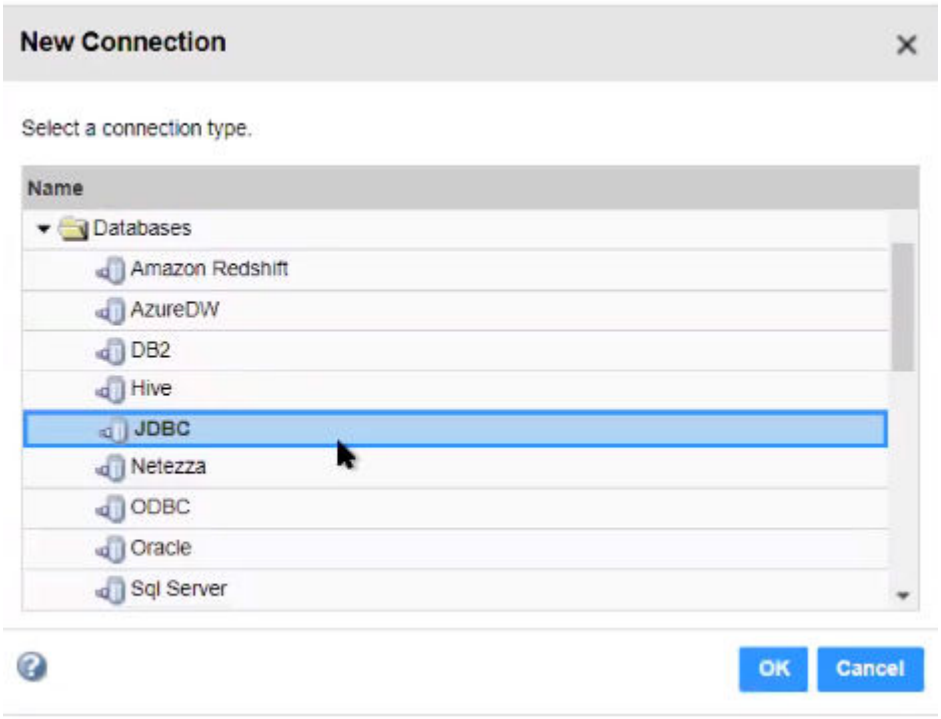
   The following image shows the New Connection control:



The **New Connection** dialog box opens.

3. Expand the Databases section, select **JDBC**, and click **OK**.

   The following image shows the New Connection dialog box:



   The **Configure New Connection** dialog box opens.

4. Configure the following connection properties:

Property	Description
Name	Type a name for the connection. Example: DatabricksCxn
ID	Default: Automatically populated with the connection name. Changing this default value is optional.
Username	Type the following value to use the Databricks user token: `token`

Property	Description
Password	Value of the token that you configured for the Databricks user. See "Prepare a Databricks User for Token Authentication" on page 11
JDBC Driver Class Name	Type the following value: `com.simba.spark.jdbc4.Driver`
Connection String	Connection to the Databricks resource. This connection string contains all the information that the domain needs to connect to the resource. The connection string contains the following elements: - jdbc:spark://<server host name>. - Port number. - Transport mode. - ssl. Use `1` to enable SSL. - httpPath. - UID. User ID that will be used to run jobs on the cluster. Use `token`. - PWD. Value of the token that you configured for the Databricks user. See "Prepare a Databricks User for Token Authentication" on page 11. Example: `jdbc:spark://abc-a1b2c3d4-cf1b.cloud.databricks.com:443/default;transportMode=http;ssl=1;httpPath=sql/protocolv1/o/5275227018426481/1234-567890-blued456;AuthMech=3;UID=token;PWD=<personal-access-token>` To get the value of these parameters from the Advanced Options area of the cluster configuration settings: 1. In the Databricks environment, select Clusters. 2. Select the cluster to connect to. 3. Expand the Advanced Options and click the **JDBC/ODBC** tab. For more information about the JDBC connection string for Databricks, see the Databricks documentation.

5. Click **Finish**.

Databricks Connection Properties

Use the Databricks connection to run mappings on a Databricks cluster.

A Databricks connection is a cluster type connection. You can create and manage a Databricks connection in the Administrator tool or the Developer tool. You can use infacmd to create a Databricks connection. Configure properties in the Databricks connection to enable communication between the Data Integration Service and the Databricks cluster.

The following table describes the general connection properties for the Databricks connection:

Property	Description
Name	The name of the connection. The name is not case sensitive and must be unique within the domain. You can change this property after you create the connection. The name cannot exceed 128 characters, contain spaces, or contain the following special characters:~ ` ! $ % ^ & * ( ) - + = { [}] | \ : ; " ' < , > . ? /
ID	String that the Data Integration Service uses to identify the connection. The ID is not case sensitive. It must be 255 characters or less and must be unique in the domain. You cannot change this property after you create the connection. Default value is the connection name.
Description	Optional. The description of the connection. The description cannot exceed 4,000 characters.

Property	Description
Connection Type	Choose Databricks.
Cluster Configuration	Name of the cluster configuration associated with the Databricks environment. Required if you do not configure the cloud provisioning configuration.
Cloud Provisioning Configuration	Name of the cloud provisioning configuration associated with a Databricks cloud platform. Required if you do not configure the cluster configuration.
Staging Directory	The directory where the Databricks Spark engine stages run-time files. If you specify a directory that does not exist, the Data Integration Service creates it at run time. If you do not provide a directory path, the run-time staging files are written to */<cluster staging directory>/DATABRICKS*.
Advanced Properties	List of advanced properties that are unique to the Databricks environment. You can configure run-time properties for the Databricks environment in the Data Integration Service and in the Databricks connection. You can override a property configured at a high level by setting the value at a lower level. For example, if you configure a property in the Data Integration Service custom properties, you can override it in the Databricks connection. The Data Integration Service processes property overrides based on the following priorities: 1. Databricks connection advanced properties 2. Data Integration Service custom properties **Note:** Informatica does not recommend changing these property values before you consult with third-party documentation, Informatica documentation, or Informatica Global Customer Support. If you change a value without knowledge of the property, you might experience performance degradation or other unexpected results.

Advanced Properties

Configure the following properties in the **Advanced Properties** of the Databricks configuration section:

infaspark.json.parser.mode

Specifies the parser how to handle corrupt JSON records. You can set the value to one of the following modes:

- DROPMALFORMED. The parser ignores all corrupted records. Default mode.
- PERMISSIVE. The parser accepts non-standard fields as nulls in corrupted records.
- FAILFAST. The parser generates an exception when it encounters a corrupted record and the Spark application goes down.

infaspark.json.parser.multiLine

Specifies whether the parser can read a multiline record in a JSON file. You can set the value to true or false. Default is false. Applies only to non-native distributions that use Spark version 2.2.x and above.

infaspark.flatfile.writer.nullValue

When the Databricks Spark engine writes to a target, it converts null values to empty strings (" "). For example, 12, AB,"",23p09udj.

The Databricks Spark engine can write the empty strings to string columns, but when it tries to write an empty string to a non-string column, the mapping fails with a type mismatch.

To allow the Databricks Spark engine to convert the empty strings back to null values and write to the target, configure the property in the Databricks Spark connection.

Set to: TRUE

infaspark.pythontx.exec

Required to run a Python transformation on the Databricks Spark engine. Set to the location of the Python executable binary on the worker nodes in the Databricks cluster.

When you provision the cluster at run time, set this property in the Databricks cloud provisioning configuration. Otherwise, set on the Databricks connection.

For example, set to:

```
infaspark.pythontx.exec=/databricks/python3/bin/python3
```

infaspark.pythontx.executorEnv.PYTHONHOME

Required to run a Python transformation on the Databricks Spark engine. Set to the location of the Python installation directory on the worker nodes in the Databricks cluster.

When you provision the cluster at run time, set this property in the Databricks cloud provisioning configuration. Otherwise, set on the Databricks connection.

For example, set to:

```
infaspark.pythontx.executorEnv.PYTHONHOME=/databricks/python3
```

Author

Mark Pritchard